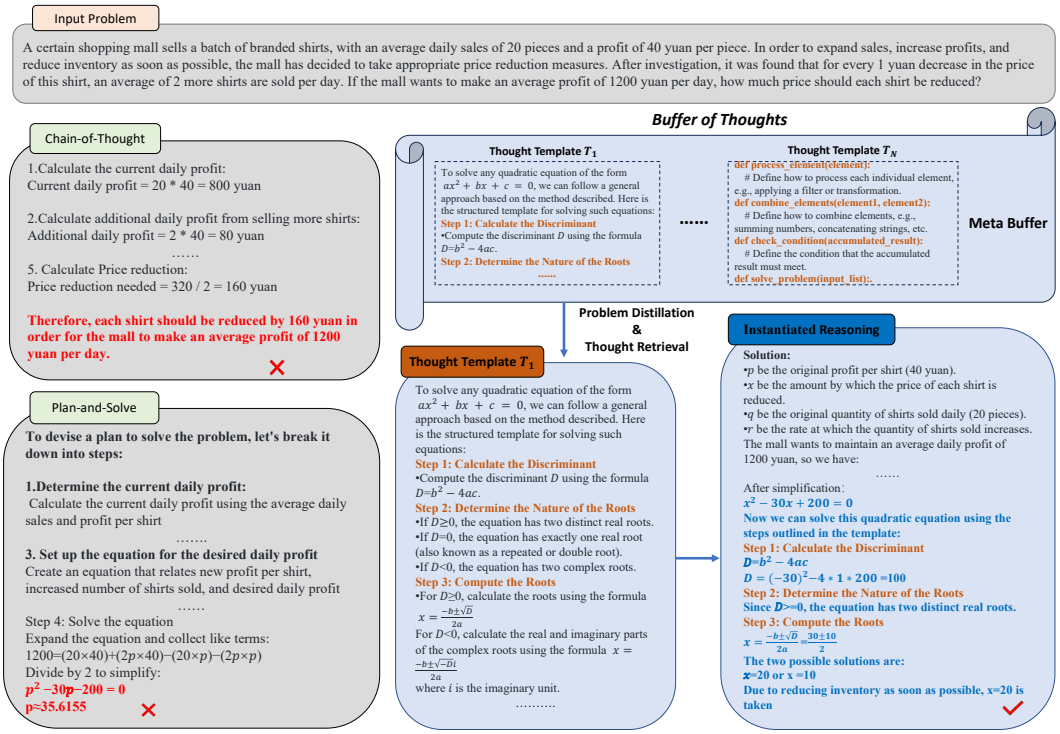

Figure 2: Illustration of different reasoning process. Buffer of Thoughts enables large language models to tackle complex reasoning tasks through our thought-augmented reasoning process. Thought template is marked in orange and instantiated thought is marked in blue.

concretely, we design a meta prompt ϕ to first distill and formalize the task information. The distilled task information could be denoted as:

$$x_d = LLM(\phi(x)), \tag{1}$$

where x is the task statement. Due to the page limit, we put the detailed meta prompt for problem-distiller in Appendix A.2.

Problem Condensation and Translation We use the problem distiller to extract key elements from input tasks, focusing on: (1). Essential parameters and variables for problem-solving; (2). The objectives of the input tasks and their corresponding constraints. We then re-organize this distilled information into a clear, comprehensible format for the subsequent reasoning stage. We then translate the specific problems into high-level concepts and structures. This translation procedure decomposes complex real-world problems, like intricate mathematical application scenarios, into simpler, multi-step calculations, making it easier for later retrieval of high-level thought.

3.2 Thought-Augmented Reasoning with Meta Buffer

Motivation Human often summarize and induce higher-level guidelines when solving problems and then apply them to relevant problems. Motivated by this, we propose *meta-buffer*, a lightweight library that contains a series of high-level thoughts (*thought-template*) for addressing various types of problems. Unlike traditional methods [11, 46, 12, 36, 9] that require specific instructions or exemplars, our high-level thought-templates can be adaptively instantiated when solving different problems, thereby enhancing LLMs with superior precision and flexibility.

Thought Template As a kind of high-level guideline, our thought-template is stored in meta-buffer , and is obtained from various problem-solving processes by our *buffer-manager*. The details about acquiring thought-templates would be introduced in Section 3.3. Since our BoT aims to provide a general reasoning approach for various tasks, we correspondingly classify the thought-templates into six categories: Text Comprehension, Creative Language Generation, Common Sense Reasoning, Mathematical Reasoning, Code Programming and Application Scheduling. We provide some example thought-templates in Appendix A.1. Such classification of thought-templates can

facilitate the template retrieval for finding most suitable solutions to different problems. Here we denote thought template, template description and its corresponding category as (T_i, D_{T_i}, C_k), where i denotes the index of meta-template, $k \in \mathbb{Z}^+$ and $1 \leq k \leq 6$, which means C_k is in one of the six categories, and D_{T_i} is the description of thought template.

Template Retrieval For each task, our BoT retrieves a thought-template T_i that is highly similar to the distilled problem x_d by calculating the embedding similarity between the description D_{T_i} and x_d. The retrieval process can be formulated as:

$$j = \text{argmax}_i(\text{Sim}(f(x_d), \{f(D_{T_i})\}_{i=1}^{N})), \quad \text{where} \quad \text{Sim}(f(x_d), \{f(D_{T_i})\}_{i=0}^{n}) >= \delta, \tag{2}$$

N is the size of the meta-buffer, $f(\cdot)$ is a normal text embedding model, and T_j denotes the retrieved thought template. We set a threshold δ (0.5$\sim$0.7 is recommended) to determine whether the current task is new. Therefore, if $\text{Sim}(f(x_d), \{f(D_{T_i})\}_{i=0}^{n}) < \delta$, we identify the task x as a new task.

Instantiated Reasoning For each specific task, we discuss two situations for the instantiated reasoning, depending on whether the current task is new: The first situation is that we successfully retrieve a thought-template T_j for the task. In this case, as presented in Figure 2, our thought-augmented reasoning will be adaptively instantiated to suitable reasoning structures with our designed instantiation prompt (in Appendix A.3). For example, in a Checkmate-in-One problem, we instantiate the template of updating chess board state to solve the problem step by step. Thus we conduct the instantiated reasoning for task x using the distilled information x_d and the retrieved template T_j, and produce its solution S_x as:

$$S_x = LLM_{\text{instantiation}}(x_d, T_j), \tag{3}$$

where $LLM_{\text{instantiation}}$ denotes the instantiated reasoner with a LLM.

In the second situation, the task is identified as a new task. To enable proper instantiated reasoning, we prepare three general coarse-grained thought-templates for utilization. Based on the distilled task information x_d, our BoT would automatically assign a suitable thought-template to the reasoning process. The detailed pre-defined thought-templates are included in Appendix A.3).

3.3 Buffer Manager

We propose *buffer-manager* to summarize the high-level guidelines and thoughts that are gained from each problem-solving process. It can generalize each specific solution to more problems, storing the critical distilled knowledge in the form of thought-templates within the meta buffer. In contrast to methods that **temporarily** generate exemplars or instructions for each problem, our buffer-manager can ensure **permanent** advancements in accuracy, efficiency, and robustness for LLM-based reasoning.

Template Distillation To extract a general though-template, we propose a three-step approach: (1) Core task summarization: identifying and describing basic types and core challenges of problems; (2) Solution steps description: summarize the general steps for solving a problem; (3) General answering template: based on the above analysis, propose a solution template or approach that can be widely applied to similar problems. Additionally, to boost the generalization ability and stability of template distillation, we carefully design two types of in-context examples of how to generate thought-template—*in-task* and *cross-task* examples. *Cross-task* means we choose the template distilled from one task to tackle the problem of other tasks, such as addressing a mathematical problem with a code-related thought-template. The new template distilled from input task x can be denoted as:

$$T_{new} = LLM_{\text{distill}}(x_d, S_x), \tag{4}$$

where LLM_{distill} is the LLM-based template distiller initialized with the following prompt:

Prompt for Template Distillation:
User: [Problem Description] + [Solution Steps or Code]
To extract and summarize the high-level paradigms and general approaches for solving such problems, please follow these steps in your response:
1. Core task summarization:
Identify and describe the basic type and core challenges of the problem, such as classifying it as a mathematical problem (e.g., solving a quadratic equation), a data structure problem (e.g., array sorting), an algorithm problem (e.g., search algorithms), etc. And analyze the most efficient way to solve the problem.
2. Solution Steps Description:
Outline the general solution steps, including how to define the problem, determine variables, list key equations or constraints, choose appropriate solving strategies and methods, and how to verify the correctness of the results.
3. General Answer Template:
Based on the above analysis, propose a template or approach that can be widely applied to this type of problem, including possible variables, functions, class definitions, etc. If it is a programming problem, provide a set of base classes and interfaces that can be used to construct solutions to specific problems.
Please ensure that your response is highly concise and structured, so that specific solutions can be transformed into generalizable methods.
[Optional] Here are some exemplars of the thought-template: *(Choose cross-task or in-task exemplars based on the analysis of the* ***Core task summarization****.)*

Dynamic Update of Meta-Buffer After template distillation, we need to consider whether the distilled template should be updated into the meta-buffer. If we initialize an empty meta-buffer or encounter a problem without a proper thought-template, the distilled thought-templates will be directly stored in the meta-buffer. If we solve problem with a retrieved thought-template, new insights may arise during the instantiation of a certain thought-template. Therefore, to avoid the redundancy of the meta-buffer while maintaining newly-generated informative thoughts, we will calculate the similarity between the embedding vectors of $D_{T_{new}}$ and $\{D_{T_i}\}_{i=0}^{n}$ and update the meta-buffer with the following rule:

$$\text{Max}(\text{Sim}(f(D_{T_{new}}), \{f(D_{T_i})\}_{i=0}^{n})) < \delta. \tag{5}$$

Otherwise, it means the meta-buffer has already possessed the necessary knowledge to solve this task and does not need to perform the update. Our dynamic update strategy effectively reduces the computational burden of template retrieval while ensuring the lightweight property of our meta-buffer. We further conduct ablation study to analyze it in Section 6.

4 Experiments

Datasets and Tasks To evaluate the efficacy of our proposed Buffer of Thoughts and compare with previous methods, we consider a diverse set of tasks and datasets that require varying degrees of mathematical and algorithmic reasoning, domain-specific knowledge, and literary creativity: (a). The **Game of 24** from ToT [14], where the objective is to form an arithmetic expression that equals 24 using each of four given numbers exactly once; (b). Three BIG-Bench Hard (BBH) [35] tasks: **Geometric Shapes**, **Multi-Step Arithmetic Two**, and **Word Sorting**; (c). Three reasoning tasks directly obtained from the BIG-Bench suite [50]: **Checkmate-in-One**, **Penguins**—where the task is to answer questions about penguins' attributes based on a given table and additional natural language information, and **DateUnderstanding**—a task that involves inferring dates from natural language descriptions, performing arithmetic operations on dates, and utilizing global knowledge such as the number of days in February; (d). **Python Programming Puzzles** (P3) [51, 52], a collection of challenging programming puzzles written in Python with varying difficulty levels; (e). **Multilingual Grade School Math** (MGSM) [33], a multilingual version of the GSM8K dataset [53] featuring translations of a subset of examples into ten typologically diverse languages, including Bengali, Japanese, and Swahili; (f). **Shakespearean Sonnet Writing** from meta-prompting [15], a novel task where the goal is to write a sonnet following the strict rhyme scheme "ABAB CDCD EFEF GG" and incorporating three provided words verbatim.